

Relatório Técnico do Sistema MyFood

Autor: Victor André Lopes Brasileiro

Projeto: MyFood 2026.1

Disciplina: Programação 2 - UFAL/IC

Tecnologia principal: Java puro com testes de aceitação via EasyAccept

Este documento descreve o sistema de forma completa, com foco em arquitetura, organização interna, responsabilidades, fluxos de execução, persistência, tratamento de exceções e padrões de projeto. Ele não foi escrito como guia rápido de uso. A intenção aqui é registrar como o sistema foi projetado, como as partes conversam e quais decisões técnicas sustentam o funcionamento das user stories. Por isso o texto entra em detalhes que normalmente não aparecem em um README.

Conteúdo do Relatório

- Descrição geral do design arquitetural do sistema.
- Principais componentes e suas interações.
- Seções específicas para cada padrão de projeto adotado.
- Para cada padrão de projeto: nome, descrição geral, problema resolvido, identificação da oportunidade e aplicação no projeto.
- Análise detalhada dos pacotes, classes, responsabilidades e fluxos principais.
- Detalhamento da persistência em XML e do tratamento de falhas de leitura e escrita.
- Rastreabilidade das user stories implementadas e dos testes de aceitação usados como validação.
- Conclusão técnica sobre a organização atual do sistema.

Descrição geral do design arquitetural do sistema

O MyFood foi construído como uma aplicação de domínio simples, executada localmente e orientada por testes de aceitação. A aplicação não usa interface gráfica, servidor HTTP, banco de dados relacional ou

framework de injeção de dependência. Mesmo assim, o sistema precisa separar entrada, regra de negócio, persistência e representação dos dados. A decisão arquitetural principal foi organizar o código em camadas pequenas, explícitas e fáceis de rastrear.

A camada de entrada é representada pela Facade. A camada de regra de negócio é representada pelos Services. A camada de acesso a dados é representada pelos Repositories. A camada de domínio é representada pelos Models.

O pacote de Exceptions funciona como apoio transversal, porque as falhas esperadas fazem parte do contrato observado pelos testes. A Facade concentra a interface pública consumida pelo EasyAccept.

Ela não decide se um email é válido, se um produto pode entrar no pedido ou se um entregador está disponível.

Essas decisões ficam nos services, que são as classes responsáveis por interpretar as regras do domínio.

Os services coordenam validações, consultas e alterações de estado. Os repositories guardam coleções em memória e resolvem a persistência em XML. Os models carregam os atributos das entidades e aplicam polimorfismo nos pontos em que a própria entidade deve responder por suas características. O fluxo de controle geralmente começa no teste, passa pela Facade, entra em um service, consulta ou altera um repository e termina alterando um model.

Quando o sistema é encerrado, os dados em memória são serializados em arquivos XML. Quando uma nova Facade é criada, os repositories tentam reconstruir o estado anterior a partir desses arquivos. O desenho evita que as regras de negócio dependam diretamente do formato XML. O desenho também evita que o EasyAccept conheça a estrutura interna do projeto.

Isso reduz acoplamento e melhora a manutenção, mesmo em um sistema pequeno. A separação entre erro de negócio e erro de persistência também faz parte do design. As exceções de negócio são classes específicas no pacote de exceptions e continuam derivando de Exception, preservando a interface esperada

pelos testes. Falhas de arquivo e leitura aparecem nos métodos de persistência por meio de IOException, enquanto as regras de negócio continuam usando exceções próprias do domínio.

Essa diferença permite entender se a falha veio de uma regra violada ou de um problema ao recuperar dados salvos. Em termos de estilo arquitetural, o sistema usa uma variação simples de arquitetura em camadas. A camada superior recebe comandos externos. A camada intermediária interpreta regras. A camada inferior guarda dados. A camada de domínio mantém o estado das entidades. Esse arranjo é suficiente para as oito user stories do projeto sem criar complexidade artificial. Ao mesmo tempo, ele deixa espaço para evolução.

Trocar XML por outro formato afetaria principalmente os repositories. Adicionar uma nova regra de pedido afetaria principalmente PedidoService. Adicionar uma nova especialização de empresa afetaria principalmente os models de empresa e os pontos de criação. A arquitetura foi escolhida para manter o projeto compreensível e para impedir que uma mudança pequena obrigue alteração em todos os arquivos.

Principais componentes e suas interações

Componente	Papel arquitetural	Interações principais
Facade	Porta pública do sistema	Recebe chamadas do EasyAccept, instancia dependências, chama services e coordena carga/salvamento
UsuarioService	Regras de usuários	Usa UsuarioRepository e os models Cliente, DonoDeEmpresa e Entregador
EmpresaService	Regras de empresas	Usa EmpresaRepository, UsuarioService e os models Restaurante, Mercado e Farmacia
ProdutoService	Regras de produtos	Usa ProdutoRepository e EmpresaService

Componente	Papel arquitetural	Interações principais
PedidoService	Regras de pedidos	Usa PedidoRepository, UsuarioService, EmpresaService e ProdutoService
EntregaService	Regras de entregas	Usa EntregaRepository, PedidoService, UsuarioService, EmpresaService e ProdutoService
Repositories	Coleções e persistência	Salvam, carregam e consultam objetos de domínio
Models	Estado do domínio	Guardam atributos, expõem getters/setters e respondem atributos específicos
Exceptions	Contrato de falhas	Representam erros de validação e estados inválidos esperados pelos testes

Fluxo: Criação de usuário cliente

1. Facade recebe criarUsuario com quatro parâmetros.
2. UsuarioService valida nome, email, senha e endereço.
3. UsuarioRepository verifica se o email já existe.
4. UsuarioRepository gera um id sequencial.
5. UsuarioService cria um Cliente.
6. UsuarioRepository guarda o objeto na lista de usuários.

Interação observada:

- A Facade mantém a entrada externa estável.
- O service concentra a decisão de regra.
- O repository participa quando a operação depende de armazenamento ou consulta.
- O model mantém os dados resultantes da operação.

Fluxo: Criação de dono de empresa

1. Facade recebe criarUsuario com CPF.
2. UsuarioService aplica as validações comuns.
3. UsuarioService valida a estrutura do CPF.
4. UsuarioRepository garante que o email não está duplicado.
5. UsuarioService cria um DonoDeEmpresa.

6. O CPF passa a ser atributo próprio do model.

Interação observada:

- A Facade mantém a entrada externa estável.
- O service concentra a decisão de regra.
- O repository participa quando a operação depende de armazenamento ou consulta.
- O model mantém os dados resultantes da operação.

Fluxo: Criação de entregador

1. Facade recebe criarUsuario com veículo e placa.
2. UsuarioService valida os dados comuns.
3. UsuarioService valida veículo e placa.
4. UsuarioRepository verifica placa duplicada usando comportamento polimórfico do usuário.
5. UsuarioService cria Entregador.
6. O entregador fica disponível para ser associado a empresas.

Interação observada:

- A Facade mantém a entrada externa estável.
- O service concentra a decisão de regra.
- O repository participa quando a operação depende de armazenamento ou consulta.
- O model mantém os dados resultantes da operação.

Fluxo: Criação de empresa

1. Facade encaminha criarEmpresa para EmpresaService.
2. EmpresaService confere se o usuário dono realmente pode criar empresa.
3. EmpresaService valida campos comuns e específicos do tipo solicitado.
4. EmpresaRepository gera id.
5. EmpresaService cria Restaurante, Mercado ou Farmacia.
6. EmpresaRepository guarda a empresa em memória.

Interação observada:

- A Facade mantém a entrada externa estável.
- O service concentra a decisão de regra.
- O repository participa quando a operação depende de armazenamento ou consulta.
- O model mantém os dados resultantes da operação.

Fluxo: Criação de produto

1. Facade chama ProdutoService.
2. ProdutoService valida nome, valor e categoria.
3. ProdutoService verifica duplicidade de nome dentro da mesma empresa.
4. ProdutoRepository gera id.
5. ProdutoRepository guarda o produto.
6. O produto passa a ser localizável pelo id ou pelo par nome/empresa.

Interação observada:

- A Facade mantém a entrada externa estável.
- O service concentra a decisão de regra.
- O repository participa quando a operação depende de armazenamento ou consulta.
- O model mantém os dados resultantes da operação.

Fluxo: Criação de pedido

1. Facade chama PedidoService.
2. PedidoService confere se o usuário não é dono de empresa.
3. PedidoService verifica se já existe pedido aberto para cliente e empresa.
4. PedidoRepository gera número.
5. Pedido nasce com estado aberto.
6. PedidoRepository guarda o pedido.

Interação observada:

- A Facade mantém a entrada externa estável.
- O service concentra a decisão de regra.
- O repository participa quando a operação depende de armazenamento ou consulta.
- O model mantém os dados resultantes da operação.

Fluxo: Fechamento e liberação de pedido

1. fecharPedido muda o estado para preparando.
2. Pedido preparando não aceita adicionar nem remover produto.
3. liberarPedido só funciona quando o pedido está preparando.
4. Pedido liberado passa para pronto.
5. Pedido pronto pode ser obtido por entregador válido.
6. Esse fluxo protege a ordem da preparação.

Interação observada:

- A Facade mantém a entrada externa estável.
- O service concentra a decisão de regra.
- O repository participa quando a operação depende de armazenamento ou consulta.
- O model mantém os dados resultantes da operação.

Fluxo: Criação e finalização de entrega

1. EntregaService recebe pedido, entregador e destino.
2. O pedido precisa estar pronto.
3. O usuário precisa ser entregador.
4. O entregador não pode estar em entrega ativa.
5. EntregaRepository gera id e guarda a entrega.
6. PedidoService marca o pedido como entregando.
7. Ao entregar, PedidoService marca o pedido como entregue.

Interação observada:

- A Facade mantém a entrada externa estável.
- O service concentra a decisão de regra.
- O repository participa quando a operação depende de armazenamento ou consulta.
- O model mantém os dados resultantes da operação.

Fluxo: Persistência

1. Facade cria repositories na inicialização.
2. Cada repository tenta carregar seu XML.
3. Arquivo inexistente é tratado como ausência normal de dados.
4. A leitura dos arquivos fica concentrada no repository responsável por cada conjunto de dados.
5. encerrarSistema chama salvarDados em todos os services.
6. A escrita dos arquivos fica concentrada no encerramento do sistema.

Interação observada:

- A Facade mantém a entrada externa estável.
- O service concentra a decisão de regra.
- O repository participa quando a operação depende de armazenamento ou consulta.
- O model mantém os dados resultantes da operação.

Padrão de Projeto: Facade

Nome do Padrão de Projeto

Facade.

Descrição Geral

Facade é um padrão estrutural que oferece uma interface única e simplificada para um conjunto de classes

internas. A classe fachada fica na fronteira entre o cliente externo e a implementação interna. O cliente

externo não precisa conhecer a quantidade de objetos envolvidos, a ordem de inicialização ou as dependências entre eles. A fachada expõe operações de alto nível, normalmente com nomes próximos dos casos de uso.

Internamente, ela delega o trabalho para classes especializadas. O objetivo do padrão não é esconder

regra mal organizada dentro de uma classe grande. O objetivo é fornecer um ponto de entrada estável para

um subsistema que possui várias partes. Quando bem aplicado, Facade reduz acoplamento, facilita teste

externo e protege o cliente contra mudanças internas.

Problema Resolvido

O problema resolvido no MyFood é a necessidade de ter uma única entrada pública sem transformar essa

entrada em uma classe cheia de regra. O EasyAccept trabalha chamando métodos de uma classe indicada nos

scripts. Se os testes tivessem que instanciar UsuarioService, EmpresaService, ProdutoService, PedidoService e EntregaService, a suíte ficaria dependente da organização interna. Se cada teste precisasse montar repositories e conectar services, a entrada do sistema deixaria de ser simples.

Também seria mais difícil alterar a estrutura interna sem alterar todos os scripts. A Facade resolve isso

criando uma barreira entre os testes e a implementação. Os scripts chamam métodos de negócio em

uma

classe só. A classe só repassa a chamada para quem realmente sabe executar a regra.

Identificação da Oportunidade

A oportunidade surgiu logo na forma como o projeto é avaliado. O EasyAccept exige uma classe de fachada prática, porque os comandos dos arquivos de teste precisam bater com métodos públicos. Ao mesmo tempo, as user stories adicionam regras diferentes: contas, empresas, produtos, pedidos, mercados, farmácias, entregadores e entregas. Misturar todas essas regras na classe chamada pelo EasyAccept deixaria o sistema difícil de ler.

A presença de várias áreas de negócio indicou que a classe de entrada deveria ser apenas uma interface de acesso. A criação dos services dentro da Facade também indicou a necessidade de centralizar a montagem das dependências. Por isso o padrão se encaixou naturalmente: uma classe pública de entrada e várias classes internas especializadas.

Aplicação no Projeto

A aplicação concreta do padrão está em `src/br/ufal/ic/myfood/Facade.java`. A Facade cria `UsuarioRepository` e `UsuarioService`. Depois cria `EmpresaRepository` e `EmpresaService`, passando `UsuarioService` como dependência. Depois cria `ProdutoRepository` e `ProdutoService`, passando `EmpresaService`.

Depois cria `PedidoRepository` e `PedidoService`, passando `UsuarioService`, `EmpresaService` e `ProdutoService`.

Por último cria `EntregaRepository` e `EntregaService`, porque entrega depende das regras de pedido, usuário, empresa e produto. Essa ordem de construção reflete as dependências reais do domínio. A Facade também chama `carregarDados` para cada repository durante a construção.

No encerramento, chama `salvarDados` para cada service. Os métodos públicos seguem os nomes usados pelos testes, como `criarUsuario`, `criarEmpresa`, `criarProduto`, `criarPedido`, `criarEntrega`, `getAtributoUsuario` e

getAtributoEmpresa. Exemplo prático: criarPedido na Facade não sabe validar se o usuário é dono de empresa. Essa validação pertence ao PedidoService.

Exemplo prático: alterarFuncionamento na Facade não sabe se o id é de um mercado. Essa decisão pertence

ao EmpresaService e ao comportamento do model Empresa. Exemplo prático: encerrarSistema não grava XML

diretamente. Ele chama os services, que chamam os repositories responsáveis.

A Facade, portanto, é a porta de entrada, não o lugar onde o sistema inteiro acontece.

Padrão de Projeto: Repository

Nome do Padrão de Projeto

Repository.

Descrição Geral

Repository é um padrão que encapsula o acesso a dados e apresenta esse acesso como se fosse uma coleção

de objetos de domínio. O código de regra de negócio não precisa saber se os dados estão em arquivo, banco

ou memória. Ele pede operações como salvar, buscar por id, listar todos ou consultar por algum critério.

O repository traduz essas operações para a forma concreta de armazenamento.

Em sistemas maiores, o padrão costuma ficar entre a camada de domínio e a infraestrutura de banco de

dados. No MyFood, ele fica entre os services e os arquivos XML. A estrutura é direta: cada entidade principal tem um repository específico. Isso evita uma classe genérica demais e deixa cada arquivo de

persistência claramente associado a uma parte do domínio.

Problema Resolvido

O problema resolvido é a mistura entre regra de negócio e persistência. Validar senha não tem relação com

escrever XML. Impedir pedido duplicado não tem relação com abrir FileInputStream. Priorizar pedidos

de
farmácia não tem relação com XMLDecoder.

Quando essas tarefas ficam juntas, a classe cresce em direções diferentes e passa a ter motivos demais para mudar. O Repository separa essas preocupações. Os services tratam do que pode ou não pode acontecer no domínio. Os repositories tratam de onde os objetos ficam e de como são recuperados.

Identificação da Oportunidade

A oportunidade apareceu pela própria presença de métodos salvarDados e carregarDados. Esses métodos precisam existir para a execução dos testes e para manter estado entre execuções. Mas eles não pertencem ao mesmo tipo de raciocínio de validar usuário, empresa, produto, pedido ou entrega. O sistema também possui cinco conjuntos de dados independentes: usuários, empresas, produtos, pedidos e entregas. Cada conjunto precisa de lista própria, id próprio e arquivo próprio. Essa repetição organizada indicou que a persistência deveria ser isolada em classes próprias. Assim, cada service usa um repository sem se comprometer com o detalhe do XML.

Aplicação no Projeto

O padrão foi aplicado com cinco repositories. UsuarioRepository mantém usuários, controla o próximo id, busca por id, busca login, verifica email e verifica placa. EmpresaRepository mantém empresas, controla o próximo id, busca por id e lista todas. ProdutoRepository mantém produtos, controla o próximo id, busca por id, busca por nome e empresa e lista por empresa.

PedidoRepository mantém pedidos, controla o próximo número, busca por número, busca pedido aberto e lista por cliente e empresa. EntregaRepository mantém entregas, controla o próximo id, busca por id, busca por pedido e lista todas. Cada repository usa XMLEncoder para salvar e XMLDecoder para carregar.

Cada arquivo

grava primeiro o próximo id ou número e depois a lista de objetos.

Essa ordem permite retomar a sequência correta depois de uma nova inicialização. A carga de dados declara

IOException. Arquivo inexistente é considerado situação normal. Erro de leitura ou permissão pertence ao

fluxo de persistência, não ao fluxo de regra de negócio.

A separação facilita localizar a diferença entre problema de arquivo e problema de regra do sistema.

Análise detalhada dos componentes

Facade

Camada: Entrada Arquivo: `src/br/ufal/ic/myfood/Facade.java` Papel principal: centralizar a interface pública do sistema. Entrada típica: comandos do EasyAccept. Saída típica: chamadas aos services ou mensagens de persistência. Dependências principais: todos os services e repositories.

Dados mantidos ou manipulados: referências para services. Responsabilidades observadas:

- constrói o grafo de dependências.
- mantém os nomes públicos usados nos testes.
- trata IOException de carga e salvamento.
- recria o sistema em `zerarSistema`.
- não valida regras de negócio.

Análise técnica:

- Facade fica em uma posição bem definida da arquitetura e não precisa assumir tarefas de outras camadas.
- Quando Facade participa de um fluxo, a chamada vem de uma camada superior ou de uma colaboração explícita.
- A classe contribui para manter o domínio separado da persistência e da entrada dos testes.
- A manutenção fica mais simples porque o papel de Facade é identificável pelo pacote e pelos métodos expostos.
- Se uma mudança futura tocar essa classe, o impacto esperado fica principalmente dentro da camada Entrada.

Main

Camada: Execução Arquivo: src/br/ufal/ic/myfood/Main.java Papel principal: executar os scripts de teste.

Entrada típica: caminho dos testes. Saída típica: resultado do EasyAccept. Dependências principais: EasyAccept e Facade.

Dados mantidos ou manipulados: não mantém estado de domínio. Responsabilidades observadas:

- procura a pasta tests.
- roda as user stories em sequência.
- serve como ponto simples de execução.
- não interfere na arquitetura de negócio.
- ajuda quando o projeto é aberto em pastas diferentes.

Análise técnica:

- Main fica em uma posição bem definida da arquitetura e não precisa assumir tarefas de outras camadas.
- Quando Main participa de um fluxo, a chamada vem de uma camada superior ou de uma colaboração explícita.
- A classe contribui para manter o domínio separado da persistência e da entrada dos testes.
- A manutenção fica mais simples porque o papel de Main é identificável pelo pacote e pelos métodos expostos.
- Se uma mudança futura tocar essa classe, o impacto esperado fica principalmente dentro da camada Execução.

UsuarioService

Camada: Regra de negócio Arquivo: src/br/ufal/ic/myfood/services/UsuarioService.java Papel principal:

controlar criação, login e consulta de usuários. Entrada típica: dados de usuário e credenciais. Saída típica: usuários criados, id de login ou atributos consultados. Dependências principais: UsuarioRepository e models de usuário.

Dados mantidos ou manipulados: não armazena lista própria. Responsabilidades observadas:

- valida nome, email, senha e endereço.
- valida CPF para dono.
- valida veículo e placa para entregador.
- consulta papel do usuário por polimorfismo.

- converte id textual inválido em exceção de usuário não cadastrado.

Análise técnica:

- UsuarioService fica em uma posição bem definida da arquitetura e não precisa assumir tarefas de outras camadas.
- Quando UsuarioService participa de um fluxo, a chamada vem de uma camada superior ou de uma colaboração explícita.
- A classe contribui para manter o domínio separado da persistência e da entrada dos testes.
- A manutenção fica mais simples porque o papel de UsuarioService é identificável pelo pacote e pelos métodos expostos.
- Se uma mudança futura tocar essa classe, o impacto esperado fica principalmente dentro da camada Regra de negócio.

EmpresaService

Camada: Regra de negócio Arquivo: src/br/ufal/ic/myfood/services/EmpresaService.java Papel principal:

controlar empresas e vínculo com entregadores. Entrada típica: dados de empresa, dono e entregador. Saída

típica: empresa criada, atributos, listas formatadas. Dependências principais: EmpresaRepository e UsuarioService.

Dados mantidos ou manipulados: não armazena lista própria. Responsabilidades observadas:

- valida se usuário é dono.
- valida tipo de empresa.
- valida horários de mercado.
- impede duplicidades relevantes.
- mantém vínculo entre empresa e entregadores.

Análise técnica:

- EmpresaService fica em uma posição bem definida da arquitetura e não precisa assumir tarefas de outras camadas.
- Quando EmpresaService participa de um fluxo, a chamada vem de uma camada superior ou de uma colaboração explícita.
- A classe contribui para manter o domínio separado da persistência e da entrada dos testes.
- A manutenção fica mais simples porque o papel de EmpresaService é identificável pelo pacote e pelos métodos expostos.
- Se uma mudança futura tocar essa classe, o impacto esperado fica principalmente dentro da camada Regra de negócio.

ProdutoService

Camada: Regra de negócio Arquivo: src/br/ufal/ic/myfood/services/ProdutoService.java Papel principal:

controlar produtos de empresas. Entrada típica: empresa, nome, valor e categoria. Saída típica: produto

criado, produto editado, lista ou atributo. Dependências principais: ProdutoRepository e EmpresaService.

Dados mantidos ou manipulados: não armazena lista própria. Responsabilidades observadas:

- valida nome do produto.
- valida valor negativo.
- valida categoria.
- impede produto duplicado na mesma empresa.
- formata valor com duas casas decimais.

Análise técnica:

- ProdutoService fica em uma posição bem definida da arquitetura e não precisa assumir tarefas de outras camadas.
- Quando ProdutoService participa de um fluxo, a chamada vem de uma camada superior ou de uma colaboração explícita.
- A classe contribui para manter o domínio separado da persistência e da entrada dos testes.
- A manutenção fica mais simples porque o papel de ProdutoService é identificável pelo pacote e pelos métodos expostos.
- Se uma mudança futura tocar essa classe, o impacto esperado fica principalmente dentro da camada Regra de negócio.

PedidoService

Camada: Regra de negócio Arquivo: src/br/ufal/ic/myfood/services/PedidoService.java Papel principal: controlar ciclo de vida dos pedidos. Entrada típica: cliente, empresa, produto, número do pedido.

Saída

típica: pedido criado, estado alterado, valor calculado. Dependências principais: PedidoRepository, UsuarioService, EmpresaService e ProdutoService.

Dados mantidos ou manipulados: estado no model Pedido via repository. Responsabilidades observadas:

- impede dono de empresa de pedir.
- impede pedido aberto duplicado.

- valida produto pertencente à empresa.
- controla estados do pedido.
- prioriza pedido de farmácia quando entregador busca pedido.

Análise técnica:

- PedidoService fica em uma posição bem definida da arquitetura e não precisa assumir tarefas de outras camadas.
- Quando PedidoService participa de um fluxo, a chamada vem de uma camada superior ou de uma colaboração explícita.
- A classe contribui para manter o domínio separado da persistência e da entrada dos testes.
- A manutenção fica mais simples porque o papel de PedidoService é identificável pelo pacote e pelos métodos expostos.
- Se uma mudança futura tocar essa classe, o impacto esperado fica principalmente dentro da camada Regra de negócio.

EntregaService

Camada: Regra de negócio Arquivo: src/br/ufal/ic/myfood/services/EntregaService.java Papel principal:

controlar entregas. Entrada típica: pedido, entregador e destino. Saída típica: entrega criada, atributos consultados ou pedido entregue. Dependências principais: EntregaRepository, PedidoService, UsuarioService, EmpresaService e ProdutoService.

Dados mantidos ou manipulados: entregas via repository e estado de pedidos. Responsabilidades observadas:

- exige pedido pronto.
- exige entregador válido.
- impede entregador ocupado.
- usa endereço do cliente quando destino vem vazio.
- marca pedido como entregue ao finalizar.

Análise técnica:

- EntregaService fica em uma posição bem definida da arquitetura e não precisa assumir tarefas de outras camadas.
- Quando EntregaService participa de um fluxo, a chamada vem de uma camada superior ou de uma colaboração explícita.
- A classe contribui para manter o domínio separado da persistência e da entrada dos testes.
- A manutenção fica mais simples porque o papel de EntregaService é identificável pelo pacote e pelos métodos expostos.

- Se uma mudança futura tocar essa classe, o impacto esperado fica principalmente dentro da camada Regra de negócio.

UsuarioRepository

Camada: Persistência Arquivo: src/br/ufal/ic/myfood/repositories/UsuarioRepository.java Papel principal:

armazenar usuários. Entrada típica: Usuario e critérios de busca. Saída típica: usuário encontrado, booleanos de duplicidade, XML. Dependências principais: models de Usuario e java.beans.

Dados mantidos ou manipulados: lista de usuários e próximo id. Responsabilidades observadas:

- salva usuarios.xml.
- carrega usuarios.xml.
- gera ids sequenciais.
- busca por email e senha.
- usa comportamento temPlaca para evitar instanceof.

Análise técnica:

- UsuarioRepository fica em uma posição bem definida da arquitetura e não precisa assumir tarefas de outras camadas.
- Quando UsuarioRepository participa de um fluxo, a chamada vem de uma camada superior ou de uma colaboração explícita.
- A classe contribui para manter o domínio separado da persistência e da entrada dos testes.
- A manutenção fica mais simples porque o papel de UsuarioRepository é identificável pelo pacote e pelos métodos expostos.
- Se uma mudança futura tocar essa classe, o impacto esperado fica principalmente dentro da camada Persistência.

EmpresaRepository

Camada: Persistência Arquivo: src/br/ufal/ic/myfood/repositories/EmpresaRepository.java Papel principal:

armazenar empresas. Entrada típica: Empresa e id. Saída típica: empresa encontrada ou lista completa.

Dependências principais: models de Empresa e java.beans.

Dados mantidos ou manipulados: lista de empresas e próximo id. Responsabilidades observadas:

- salva empresas.xml.
- carrega empresas.xml.

- gera ids sequenciais.
- não valida dono.
- não conhece regra de mercado ou farmácia.

Análise técnica:

- EmpresaRepository fica em uma posição bem definida da arquitetura e não precisa assumir tarefas de outras camadas.
- Quando EmpresaRepository participa de um fluxo, a chamada vem de uma camada superior ou de uma colaboração explícita.
- A classe contribui para manter o domínio separado da persistência e da entrada dos testes.
- A manutenção fica mais simples porque o papel de EmpresaRepository é identificável pelo pacote e pelos métodos expostos.
- Se uma mudança futura tocar essa classe, o impacto esperado fica principalmente dentro da camada Persistência.

ProdutoRepository

Camada: Persistência Arquivo: src/br/ufal/ic/myfood/repositories/ProdutoRepository.java Papel principal:

armazenar produtos. Entrada típica: Produto, id, nome e empresa. Saída típica: produto encontrado ou

lista por empresa. Dependências principais: Produto e java.beans.

Dados mantidos ou manipulados: lista de produtos e próximo id. Responsabilidades observadas:

- salva produtos.xml.
- carrega produtos.xml.
- busca por nome e empresa.
- lista produtos por empresa.
- não calcula valor de pedido.

Análise técnica:

- ProdutoRepository fica em uma posição bem definida da arquitetura e não precisa assumir tarefas de outras camadas.
- Quando ProdutoRepository participa de um fluxo, a chamada vem de uma camada superior ou de uma colaboração explícita.
- A classe contribui para manter o domínio separado da persistência e da entrada dos testes.
- A manutenção fica mais simples porque o papel de ProdutoRepository é identificável pelo pacote e pelos métodos expostos.
- Se uma mudança futura tocar essa classe, o impacto esperado fica principalmente dentro da camada Persistência.

PedidoRepository

Camada: Persistência Arquivo: src/br/ufal/ic/myfood/repositories/PedidoRepository.java Papel principal:

armazenar pedidos. Entrada típica: Pedido, número, cliente e empresa. Saída típica: pedido encontrado ou

listas filtradas. Dependências principais: Pedido e java.beans.

Dados mantidos ou manipulados: lista de pedidos e próximo número. Responsabilidades observadas:

- salva pedidos.xml.
- carrega pedidos.xml.
- busca pedido aberto.
- lista pedidos por cliente e empresa.
- não decide transições de estado.

Análise técnica:

- PedidoRepository fica em uma posição bem definida da arquitetura e não precisa assumir tarefas de outras camadas.
- Quando PedidoRepository participa de um fluxo, a chamada vem de uma camada superior ou de uma colaboração explícita.
- A classe contribui para manter o domínio separado da persistência e da entrada dos testes.
- A manutenção fica mais simples porque o papel de PedidoRepository é identificável pelo pacote e pelos métodos expostos.
- Se uma mudança futura tocar essa classe, o impacto esperado fica principalmente dentro da camada Persistência.

EntregaRepository

Camada: Persistência Arquivo: src/br/ufal/ic/myfood/repositories/EntregaRepository.java Papel principal:

armazenar entregas. Entrada típica: Entrega, id ou pedido. Saída típica: entrega encontrada ou lista completa. Dependências principais: Entrega e java.beans.

Dados mantidos ou manipulados: lista de entregas e próximo id. Responsabilidades observadas:

- salva entregas.xml.
- carrega entregas.xml.
- busca por pedido.
- lista entregas ativas e históricas.

- não decide disponibilidade de entregador.

Análise técnica:

- EntregaRepository fica em uma posição bem definida da arquitetura e não precisa assumir tarefas de outras camadas.
- Quando EntregaRepository participa de um fluxo, a chamada vem de uma camada superior ou de uma colaboração explícita.
- A classe contribui para manter o domínio separado da persistência e da entrada dos testes.
- A manutenção fica mais simples porque o papel de EntregaRepository é identificável pelo pacote e pelos métodos expostos.
- Se uma mudança futura tocar essa classe, o impacto esperado fica principalmente dentro da camada Persistência.

Usuario

Camada: Domínio Arquivo: src/br/ufal/ic/myfood/models/Usuario.java Papel principal: base abstrata dos

usuários. Entrada típica: atributo solicitado. Saída típica: valor comum ou erro de atributo.

Dependências principais: AtributoInvalidoExc.

Dados mantidos ou manipulados: id, nome, email, senha e endereço. Responsabilidades observadas:

- define getTipo.
- responde atributos comuns.
- delega atributo específico.
- por padrão não é dono nem entregador.
- evita lógica de tipo no service.

Análise técnica:

- Usuario fica em uma posição bem definida da arquitetura e não precisa assumir tarefas de outras camadas.
- Quando Usuario participa de um fluxo, a chamada vem de uma camada superior ou de uma colaboração explícita.
- A classe contribui para manter o domínio separado da persistência e da entrada dos testes.
- A manutenção fica mais simples porque o papel de Usuario é identificável pelo pacote e pelos métodos expostos.
- Se uma mudança futura tocar essa classe, o impacto esperado fica principalmente dentro da camada Domínio.

Cliente

Camada: Domínio Arquivo: src/br/ufal/ic/myfood/models/Cliente.java Papel principal: representar usuário

consumidor. Entrada típica: dados comuns de usuário. Saída típica: tipo cliente. Dependências principais:

Usuario.

Dados mantidos ou manipulados: herda dados de Usuario. Responsabilidades observadas:

- não adiciona campos.
- participa da criação de pedidos.
- não cria empresa.
- não executa entrega.
- mantém construtor vazio para XML.

Análise técnica:

- Cliente fica em uma posição bem definida da arquitetura e não precisa assumir tarefas de outras camadas.
- Quando Cliente participa de um fluxo, a chamada vem de uma camada superior ou de uma colaboração explícita.
- A classe contribui para manter o domínio separado da persistência e da entrada dos testes.
- A manutenção fica mais simples porque o papel de Cliente é identificável pelo pacote e pelos métodos expostos.
- Se uma mudança futura tocar essa classe, o impacto esperado fica principalmente dentro da camada Domínio.

DonoDeEmpresa

Camada: Domínio Arquivo: src/br/ufal/ic/myfood/models/DonoDeEmpresa.java Papel principal: representar

usuário proprietário. Entrada típica: CPF e dados comuns. Saída típica: tipo dono e atributo cpf.

Dependências principais: Usuario.

Dados mantidos ou manipulados: cpf além dos dados comuns. Responsabilidades observadas:

- responde cpf.
- marca ehDonoDeEmpresa como verdadeiro.
- é validado antes de criar empresa.
- não deve abrir pedido.

- mantém getters e setters para XML.

Análise técnica:

- `DonoDeEmpresa` fica em uma posição bem definida da arquitetura e não precisa assumir tarefas de outras camadas.
- Quando `DonoDeEmpresa` participa de um fluxo, a chamada vem de uma camada superior ou de uma colaboração explícita.
- A classe contribui para manter o domínio separado da persistência e da entrada dos testes.
- A manutenção fica mais simples porque o papel de `DonoDeEmpresa` é identificável pelo pacote e pelos métodos expostos.
- Se uma mudança futura tocar essa classe, o impacto esperado fica principalmente dentro da camada Domínio.

Entregador

Camada: Domínio Arquivo: `src/br/ufal/ic/myfood/models/Entregador.java` Papel principal: representar usuário entregador. Entrada típica: veículo, placa e dados comuns. Saída típica: tipo entregador, veículo, placa e verificação de placa. Dependências principais: `Usuario`.

Dados mantidos ou manipulados: veículo e placa além dos dados comuns. Responsabilidades observadas:

- responde veículo.
- responde placa.
- marca `ehEntregador` como verdadeiro.
- informa se tem uma placa específica.
- participa de vínculos com empresas.

Análise técnica:

- `Entregador` fica em uma posição bem definida da arquitetura e não precisa assumir tarefas de outras camadas.
- Quando `Entregador` participa de um fluxo, a chamada vem de uma camada superior ou de uma colaboração explícita.
- A classe contribui para manter o domínio separado da persistência e da entrada dos testes.
- A manutenção fica mais simples porque o papel de `Entregador` é identificável pelo pacote e pelos métodos expostos.
- Se uma mudança futura tocar essa classe, o impacto esperado fica principalmente dentro da camada Domínio.

Empresa

Camada: Domínio Arquivo: src/br/ufal/ic/myfood/models/Empresa.java Papel principal: base abstrata das

empresas. Entrada típica: atributo solicitado ou alteração de funcionamento. Saída típica: valor comum,

valor específico ou erro. Dependências principais: AtributoInvalidoExc e MercadoInvalidoException.

Dados mantidos ou manipulados: id, nome, endereço, donold e entregadoresIds. Responsabilidades observadas:

- define getTipo.
- protege lista de entregadores contra nulo.
- responde atributos comuns.
- por padrão não altera funcionamento.
- por padrão não é farmácia.

Análise técnica:

- Empresa fica em uma posição bem definida da arquitetura e não precisa assumir tarefas de outras camadas.
- Quando Empresa participa de um fluxo, a chamada vem de uma camada superior ou de uma colaboração explícita.
- A classe contribui para manter o domínio separado da persistência e da entrada dos testes.
- A manutenção fica mais simples porque o papel de Empresa é identificável pelo pacote e pelos métodos expostos.
- Se uma mudança futura tocar essa classe, o impacto esperado fica principalmente dentro da camada Domínio.

Restaurante

Camada: Domínio Arquivo: src/br/ufal/ic/myfood/models/Restaurante.java Papel principal: representar empresa restaurante. Entrada típica: tipo de cozinha. Saída típica: tipo restaurante e atributo tipoCozinha. Dependências principais: Empresa.

Dados mantidos ou manipulados: tipoCozinha além dos dados comuns. Responsabilidades observadas:

- responde tipoCozinha.
- não tem horário de mercado.
- não tem atributos de farmácia.
- usa donold herdado.

- usa entregadoresIds herdado.

Análise técnica:

- Restaurante fica em uma posição bem definida da arquitetura e não precisa assumir tarefas de outras camadas.
- Quando Restaurante participa de um fluxo, a chamada vem de uma camada superior ou de uma colaboração explícita.
- A classe contribui para manter o domínio separado da persistência e da entrada dos testes.
- A manutenção fica mais simples porque o papel de Restaurante é identificável pelo pacote e pelos métodos expostos.
- Se uma mudança futura tocar essa classe, o impacto esperado fica principalmente dentro da camada Domínio.

Mercado

Camada: Domínio Arquivo: src/br/ufal/ic/myfood/models/Mercado.java Papel principal: representar empresa

mercado. Entrada típica: abre, fecha e tipoMercado. Saída típica: tipo mercado e atributos de funcionamento. Dependências principais: Empresa.

Dados mantidos ou manipulados: abre, fecha e tipoMercado. Responsabilidades observadas:

- responde abre.
- responde fecha.
- responde tipoMercado.
- implementa alteração de funcionamento.
- depende do service para validação prévia do horário.

Análise técnica:

- Mercado fica em uma posição bem definida da arquitetura e não precisa assumir tarefas de outras camadas.
- Quando Mercado participa de um fluxo, a chamada vem de uma camada superior ou de uma colaboração explícita.
- A classe contribui para manter o domínio separado da persistência e da entrada dos testes.
- A manutenção fica mais simples porque o papel de Mercado é identificável pelo pacote e pelos métodos expostos.
- Se uma mudança futura tocar essa classe, o impacto esperado fica principalmente dentro da camada Domínio.

Farmacia

Camada: Domínio Arquivo: `src/br/ufal/ic/myfood/models/Farmacia.java` Papel principal: representar empresa

farmácia. Entrada típica: `aberto24Horas` e `numeroFuncionarios`. Saída típica: tipo farmacia e atributos próprios. Dependências principais: Empresa.

Dados mantidos ou manipulados: `aberto24Horas` e `numeroFuncionarios`. Responsabilidades observadas:

- responde `aberto24Horas`.
- responde `numeroFuncionarios`.
- marca `ehFarmacia` como verdadeiro.
- participa da prioridade de entrega.
- não usa horário de mercado.

Análise técnica:

- Farmacia fica em uma posição bem definida da arquitetura e não precisa assumir tarefas de outras camadas.
- Quando Farmacia participa de um fluxo, a chamada vem de uma camada superior ou de uma colaboração explícita.
- A classe contribui para manter o domínio separado da persistência e da entrada dos testes.
- A manutenção fica mais simples porque o papel de Farmacia é identificável pelo pacote e pelos métodos expostos.
- Se uma mudança futura tocar essa classe, o impacto esperado fica principalmente dentro da camada Domínio.

Produto

Camada: Domínio Arquivo: `src/br/ufal/ic/myfood/models/Produto.java` Papel principal: representar item comercializado. Entrada típica: `id`, empresa, nome, valor e categoria. Saída típica: dados do produto.

Dependências principais: `Serializable`.

Dados mantidos ou manipulados: `id`, `empresald`, nome, valor e categoria. Responsabilidades observadas:

- pertence a uma empresa.
- pode ser editado pelo service.
- é usado no cálculo do pedido.
- é listado por empresa.

- mantém construtor vazio para XML.

Análise técnica:

- Produto fica em uma posição bem definida da arquitetura e não precisa assumir tarefas de outras camadas.
- Quando Produto participa de um fluxo, a chamada vem de uma camada superior ou de uma colaboração explícita.
- A classe contribui para manter o domínio separado da persistência e da entrada dos testes.
- A manutenção fica mais simples porque o papel de Produto é identificável pelo pacote e pelos métodos expostos.
- Se uma mudança futura tocar essa classe, o impacto esperado fica principalmente dentro da camada Domínio.

Pedido

Camada: Domínio Arquivo: src/br/ufal/ic/myfood/models/Pedido.java Papel principal: representar compra em

andamento ou concluída. Entrada típica: cliente, empresa e produtos. Saída típica: estado, produtos e número. Dependências principais: Serializable.

Dados mantidos ou manipulados: número, clienteld, empresald, estado e produtoslds.

Responsabilidades

observadas:

- começa aberto.
- guarda ids de produtos.
- protege lista contra nulo.
- tem estado alterado pelos services.
- é base para criação de entrega.

Análise técnica:

- Pedido fica em uma posição bem definida da arquitetura e não precisa assumir tarefas de outras camadas.
- Quando Pedido participa de um fluxo, a chamada vem de uma camada superior ou de uma colaboração explícita.
- A classe contribui para manter o domínio separado da persistência e da entrada dos testes.
- A manutenção fica mais simples porque o papel de Pedido é identificável pelo pacote e pelos métodos expostos.
- Se uma mudança futura tocar essa classe, o impacto esperado fica principalmente dentro da camada Domínio.

Entrega

Camada: Domínio Arquivo: src/br/ufal/ic/myfood/models/Entrega.java Papel principal: representar transporte de pedido. Entrada típica: pedido, entregador e destino. Saída típica: dados da entrega. Dependências principais: Serializable.

Dados mantidos ou manipulados: id, pedidoid, entregadorid e destino. Responsabilidades observadas:

- liga pedido e entregador.
- mantém destino efetivo.
- é buscada por id ou por pedido.
- continua existindo após pedido entregue.
- não calcula produtos.

Análise técnica:

- Entrega fica em uma posição bem definida da arquitetura e não precisa assumir tarefas de outras camadas.
- Quando Entrega participa de um fluxo, a chamada vem de uma camada superior ou de uma colaboração explícita.
- A classe contribui para manter o domínio separado da persistência e da entrada dos testes.
- A manutenção fica mais simples porque o papel de Entrega é identificável pelo pacote e pelos métodos expostos.
- Se uma mudança futura tocar essa classe, o impacto esperado fica principalmente dentro da camada Domínio.

Modelo de domínio e regras por entidade

Usuários

- Cliente representa quem consome o serviço.
- DonoDeEmpresa representa quem pode cadastrar empresas.
- Entregador representa quem pode assumir entregas.
- Os três compartilham id, nome, email, senha e endereço.
- Os campos específicos ficam nas subclasses.
- O papel do usuário é consultado por comportamento, não por comparação direta de classe.

Impacto arquitetural:

- A entidade guarda estado, mas não tenta controlar todo o fluxo sozinha.
- A regra de transição e validação fica no service correspondente.

- Esse arranjo mantém o model simples e compatível com a serialização XML.
- Também deixa as mensagens de erro concentradas no ponto em que a regra é avaliada.

Empresas

- Restaurante possui tipo de cozinha.
- Mercado possui horário de abertura, horário de fechamento e tipo de mercado.
- Farmacia possui informação de abertura 24 horas e número de funcionários.
- Todas as empresas têm dono e lista de entregadores.
- A lista de entregadores guarda ids de usuários.
- A prioridade de farmácia aparece no fluxo de obtenção de pedido.

Impacto arquitetural:

- A entidade guarda estado, mas não tenta controlar todo o fluxo sozinha.
- A regra de transição e validação fica no service correspondente.
- Esse arranjo mantém o model simples e compatível com a serialização XML.
- Também deixa as mensagens de erro concentradas no ponto em que a regra é avaliada.

Produtos

- Produto pertence a uma empresa por empresald.
- O nome é único dentro da mesma empresa.
- O valor é usado no cálculo total do pedido.
- A categoria é uma informação consultável.
- A edição altera nome, valor e categoria.
- Produto não sabe validar a empresa; isso fica no service.

Impacto arquitetural:

- A entidade guarda estado, mas não tenta controlar todo o fluxo sozinha.
- A regra de transição e validação fica no service correspondente.
- Esse arranjo mantém o model simples e compatível com a serialização XML.
- Também deixa as mensagens de erro concentradas no ponto em que a regra é avaliada.

Pedidos

- Pedido nasce aberto.
- Pedido aberto aceita adicionar e remover produtos.
- Pedido fechado passa para preparando.
- Pedido preparando não aceita mudar lista de produtos.
- Pedido pronto pode ser entregue.
- Pedido entregando bloqueia o entregador envolvido.

- Pedido entregue representa o fim do fluxo.

Impacto arquitetural:

- A entidade guarda estado, mas não tenta controlar todo o fluxo sozinha.
- A regra de transição e validação fica no service correspondente.
- Esse arranjo mantém o model simples e compatível com a serialização XML.
- Também deixa as mensagens de erro concentradas no ponto em que a regra é avaliada.

Entregas

- Entrega só nasce a partir de pedido pronto.
- Entrega liga pedido e entregador por id.
- Destino vazio usa endereço do cliente.
- Entrega ativa mantém pedido em estado entregando.
- Ao finalizar, o pedido vira entregue.
- A entrega continua consultável depois da conclusão.

Impacto arquitetural:

- A entidade guarda estado, mas não tenta controlar todo o fluxo sozinha.
- A regra de transição e validação fica no service correspondente.
- Esse arranjo mantém o model simples e compatível com a serialização XML.
- Também deixa as mensagens de erro concentradas no ponto em que a regra é avaliada.

Persistência em XML e tratamento de exceções

dados/usuarios.xml

Repository responsável: UsuarioRepository. Conteúdo gravado: próximo id e lista de usuários.

Situações

consideradas:

- arquivo inexistente no primeiro uso.
- XML corrompido.
- classe de usuário incompatível.
- lista de usuários fora do formato esperado.

Comportamento esperado:

- Se o arquivo ainda não existe, o repository mantém a coleção vazia e segue a execução.
- Se o arquivo existe mas não pode ser lido, a falha sobe como IOException.
- Se o XML existe mas não representa os objetos esperados, a falha pertence ao carregamento daquele repository.

- A Facade centraliza a chamada de carregamento de cada repository durante a inicialização.
- A separação mantém a persistência fora das regras de negócio.

dados/empresas.xml

Repository responsável: EmpresaRepository. Conteúdo gravado: próximo id e lista de empresas.

Situações

consideradas:

- arquivo inexistente no primeiro uso.
- XML corrompido.
- subclasse de empresa incompatível.
- lista de empresas fora do formato esperado.

Comportamento esperado:

- Se o arquivo ainda não existe, o repository mantém a coleção vazia e segue a execução.
- Se o arquivo existe mas não pode ser lido, a falha sobe como IOException.
- Se o XML existe mas não representa os objetos esperados, a falha pertence ao carregamento daquele repository.
- A Facade centraliza a chamada de carregamento de cada repository durante a inicialização.
- A separação mantém a persistência fora das regras de negócio.

dados/produtos.xml

Repository responsável: ProdutoRepository. Conteúdo gravado: próximo id e lista de produtos.

Situações

consideradas:

- arquivo inexistente no primeiro uso.
- XML corrompido.
- produto com campo incompatível.
- lista de produtos fora do formato esperado.

Comportamento esperado:

- Se o arquivo ainda não existe, o repository mantém a coleção vazia e segue a execução.
- Se o arquivo existe mas não pode ser lido, a falha sobe como IOException.
- Se o XML existe mas não representa os objetos esperados, a falha pertence ao carregamento daquele repository.
- A Facade centraliza a chamada de carregamento de cada repository durante a inicialização.
- A separação mantém a persistência fora das regras de negócio.

dados/pedidos.xml

Repository responsável: PedidoRepository. Conteúdo gravado: próximo número e lista de pedidos.

Situações

consideradas:

- arquivo inexistente no primeiro uso.
- XML corrompido.
- estado de pedido incompatível.
- lista de produtos de pedido nula.

Comportamento esperado:

- Se o arquivo ainda não existe, o repository mantém a coleção vazia e segue a execução.
- Se o arquivo existe mas não pode ser lido, a falha sobe como IOException.
- Se o XML existe mas não representa os objetos esperados, a falha pertence ao carregamento daquele repository.
- A Facade centraliza a chamada de carregamento de cada repository durante a inicialização.
- A separação mantém a persistência fora das regras de negócio.

dados/entregas.xml

Repository responsável: EntregaRepository. Conteúdo gravado: próximo id e lista de entregas.

Situações

consideradas:

- arquivo inexistente no primeiro uso.
- XML corrompido.
- entrega sem pedido correspondente.
- lista de entregas fora do formato esperado.

Comportamento esperado:

- Se o arquivo ainda não existe, o repository mantém a coleção vazia e segue a execução.
- Se o arquivo existe mas não pode ser lido, a falha sobe como IOException.
- Se o XML existe mas não representa os objetos esperados, a falha pertence ao carregamento daquele repository.
- A Facade centraliza a chamada de carregamento de cada repository durante a inicialização.
- A separação mantém a persistência fora das regras de negócio.

Fluxos de estado do pedido e da entrega

Estado: aberto

Descrição: estado inicial do pedido. Operações permitidas:

- adicionar produto.
- remover produto.
- consultar produtos.
- consultar valor parcial.

Operações bloqueadas ou fora do fluxo:

- criar entrega.
- marcar entregue sem passar por entrega.

Justificativa:

- O estado protege a ordem natural do pedido.
- A validação fica nos services, porque depende de regras e mensagens de erro.
- O model guarda o valor atual do estado, mas não decide sozinho quando mudar.

Estado: preparando

Descrição: estado depois de fecharPedido. Operações permitidas:

- liberar pedido.
- consultar dados do pedido.

Operações bloqueadas ou fora do fluxo:

- adicionar produto.
- remover produto.
- criar entrega antes de pronto.

Justificativa:

- O estado protege a ordem natural do pedido.
- A validação fica nos services, porque depende de regras e mensagens de erro.
- O model guarda o valor atual do estado, mas não decide sozinho quando mudar.

Estado: pronto

Descrição: estado depois de liberarPedido. Operações permitidas:

- ser obtido por entregador habilitado.

- criar entrega.

Operações bloqueadas ou fora do fluxo:

- liberar novamente.
- alterar lista de produtos.

Justificativa:

- O estado protege a ordem natural do pedido.
- A validação fica nos services, porque depende de regras e mensagens de erro.
- O model guarda o valor atual do estado, mas não decide sozinho quando mudar.

Estado: entregando

Descrição: estado depois de criarEntrega. Operações permitidas:

- consultar entrega.
- finalizar entrega.

Operações bloqueadas ou fora do fluxo:

- mesmo entregador assumir outra entrega ativa.
- voltar para preparando.

Justificativa:

- O estado protege a ordem natural do pedido.
- A validação fica nos services, porque depende de regras e mensagens de erro.
- O model guarda o valor atual do estado, mas não decide sozinho quando mudar.

Estado: entregue

Descrição: estado final do ciclo. Operações permitidas:

- manter histórico consultável.

Operações bloqueadas ou fora do fluxo:

- nova entrega para o mesmo pedido.
- mudanças de preparação.

Justificativa:

- O estado protege a ordem natural do pedido.
- A validação fica nos services, porque depende de regras e mensagens de erro.
- O model guarda o valor atual do estado, mas não decide sozinho quando mudar.

Rastreabilidade das user stories

US1 - Criação de contas

Classes envolvidas:

- Facade.
- UsuarioService.
- UsuarioRepository.
- Cliente.
- DonoDeEmpresa.
- Entregador.

Regras cobertas:

- validar dados comuns.
- validar CPF.
- validar veículo e placa.
- impedir email duplicado.
- impedir placa duplicada.
- permitir login.

Forma de validação:

- A história é validada por scripts EasyAccept correspondentes.
- Os scripts chamam a Facade e observam retorno, exceção e estado persistido.
- O sucesso depende de regra correta, mensagem correta e formatação correta.
- O comportamento interno pode mudar, desde que o contrato externo continue igual.

US2 - Criação de Restaurantes

Classes envolvidas:

- Facade.
- EmpresaService.
- EmpresaRepository.
- Restaurante.
- UsuarioService.

Regras cobertas:

- exigir dono de empresa.
- criar restaurante.
- guardar tipo de cozinha.
- impedir duplicidade indevida.

- consultar atributos.

Forma de validação:

- A história é validada por scripts EasyAccept correspondentes.
- Os scripts chamam a Facade e observam retorno, exceção e estado persistido.
- O sucesso depende de regra correta, mensagem correta e formatação correta.
- O comportamento interno pode mudar, desde que o contrato externo continue igual.

US3 - Criação de Produtos

Classes envolvidas:

- Facade.
- ProdutoService.
- ProdutoRepository.
- Produto.
- EmpresaService.

Regras cobertas:

- validar nome.
- validar valor.
- validar categoria.
- impedir duplicidade.
- editar produto.
- listar por empresa.

Forma de validação:

- A história é validada por scripts EasyAccept correspondentes.
- Os scripts chamam a Facade e observam retorno, exceção e estado persistido.
- O sucesso depende de regra correta, mensagem correta e formatação correta.
- O comportamento interno pode mudar, desde que o contrato externo continue igual.

US4 - Criação de Pedidos

Classes envolvidas:

- Facade.
- PedidoService.
- PedidoRepository.
- Pedido.
- ProdutoService.

Regras cobertas:

- impedir dono de pedir.
- impedir pedido aberto duplicado.
- adicionar produto da empresa.
- calcular valor.
- fechar pedido.
- remover produto.

Forma de validação:

- A história é validada por scripts EasyAccept correspondentes.
- Os scripts chamam a Facade e observam retorno, exceção e estado persistido.
- O sucesso depende de regra correta, mensagem correta e formatação correta.
- O comportamento interno pode mudar, desde que o contrato externo continue igual.

US5 - Criação de Mercados

Classes envolvidas:

- Facade.
- EmpresaService.
- EmpresaRepository.
- Mercado.

Regras cobertas:

- validar tipo mercado.
- validar horário.
- validar tipoMercado.
- alterar funcionamento.
- consultar abre e fecha.

Forma de validação:

- A história é validada por scripts EasyAccept correspondentes.
- Os scripts chamam a Facade e observam retorno, exceção e estado persistido.
- O sucesso depende de regra correta, mensagem correta e formatação correta.
- O comportamento interno pode mudar, desde que o contrato externo continue igual.

US6 - Criação de Farmacia

Classes envolvidas:

- Facade.
- EmpresaService.
- EmpresaRepository.

- Farmacia.
- PedidoService.

Regras cobertas:

- validar tipo farmacia.
- guardar aberto24Horas.
- guardar numeroFuncionarios.
- consultar atributos.
- priorizar pedido de farmácia.

Forma de validação:

- A história é validada por scripts EasyAccept correspondentes.
- Os scripts chamam a Facade e observam retorno, exceção e estado persistido.
- O sucesso depende de regra correta, mensagem correta e formatação correta.
- O comportamento interno pode mudar, desde que o contrato externo continue igual.

US7 - Criação de Entregador

Classes envolvidas:

- Facade.
- UsuarioService.
- EmpresaService.
- Entregador.
- Empresa.

Regras cobertas:

- criar entregador.
- validar placa.
- cadastrar entregador em empresa.
- listar entregadores.
- listar empresas do entregador.

Forma de validação:

- A história é validada por scripts EasyAccept correspondentes.
- Os scripts chamam a Facade e observam retorno, exceção e estado persistido.
- O sucesso depende de regra correta, mensagem correta e formatação correta.
- O comportamento interno pode mudar, desde que o contrato externo continue igual.

US8 - Sistema de entregas

Classes envolvidas:

- Facade.
- EntregaService.
- EntregaRepository.
- Entrega.
- PedidoService.

Regras cobertas:

- obter pedido pronto.
- validar entregador.
- criar entrega.
- bloquear entregador ocupado.
- marcar pedido entregue.

Forma de validação:

- A história é validada por scripts EasyAccept correspondentes.
- Os scripts chamam a Facade e observam retorno, exceção e estado persistido.
- O sucesso depende de regra correta, mensagem correta e formatação correta.
- O comportamento interno pode mudar, desde que o contrato externo continue igual.

Cobertura de testes de aceitação

us1_1.txt

- Script executado pelo EasyAccept através da classe Facade.
- Verifica comportamento de fora para dentro, sem acessar services diretamente.
- Exige que métodos públicos, mensagens e formatos permaneçam compatíveis.
- Ajuda a detectar regressões em regras de negócio e persistência.
- O teste representa uma fatia de uso do sistema, não apenas uma chamada isolada.

us1_2.txt

- Script executado pelo EasyAccept através da classe Facade.
- Verifica comportamento de fora para dentro, sem acessar services diretamente.
- Exige que métodos públicos, mensagens e formatos permaneçam compatíveis.
- Ajuda a detectar regressões em regras de negócio e persistência.
- O teste representa uma fatia de uso do sistema, não apenas uma chamada isolada.

us2_1.txt

- Script executado pelo EasyAccept através da classe Facade.

- Verifica comportamento de fora para dentro, sem acessar services diretamente.
- Exige que métodos públicos, mensagens e formatos permaneçam compatíveis.
- Ajuda a detectar regressões em regras de negócio e persistência.
- O teste representa uma fatia de uso do sistema, não apenas uma chamada isolada.

us2_2.txt

- Script executado pelo EasyAccept através da classe Facade.
- Verifica comportamento de fora para dentro, sem acessar services diretamente.
- Exige que métodos públicos, mensagens e formatos permaneçam compatíveis.
- Ajuda a detectar regressões em regras de negócio e persistência.
- O teste representa uma fatia de uso do sistema, não apenas uma chamada isolada.

us3_1.txt

- Script executado pelo EasyAccept através da classe Facade.
- Verifica comportamento de fora para dentro, sem acessar services diretamente.
- Exige que métodos públicos, mensagens e formatos permaneçam compatíveis.
- Ajuda a detectar regressões em regras de negócio e persistência.
- O teste representa uma fatia de uso do sistema, não apenas uma chamada isolada.

us3_2.txt

- Script executado pelo EasyAccept através da classe Facade.
- Verifica comportamento de fora para dentro, sem acessar services diretamente.
- Exige que métodos públicos, mensagens e formatos permaneçam compatíveis.
- Ajuda a detectar regressões em regras de negócio e persistência.
- O teste representa uma fatia de uso do sistema, não apenas uma chamada isolada.

us4_1.txt

- Script executado pelo EasyAccept através da classe Facade.
- Verifica comportamento de fora para dentro, sem acessar services diretamente.
- Exige que métodos públicos, mensagens e formatos permaneçam compatíveis.
- Ajuda a detectar regressões em regras de negócio e persistência.
- O teste representa uma fatia de uso do sistema, não apenas uma chamada isolada.

us4_2.txt

- Script executado pelo EasyAccept através da classe Facade.
- Verifica comportamento de fora para dentro, sem acessar services diretamente.
- Exige que métodos públicos, mensagens e formatos permaneçam compatíveis.
- Ajuda a detectar regressões em regras de negócio e persistência.
- O teste representa uma fatia de uso do sistema, não apenas uma chamada isolada.

us5_1.txt

- Script executado pelo EasyAccept através da classe Facade.
- Verifica comportamento de fora para dentro, sem acessar services diretamente.
- Exige que métodos públicos, mensagens e formatos permaneçam compatíveis.
- Ajuda a detectar regressões em regras de negócio e persistência.
- O teste representa uma fatia de uso do sistema, não apenas uma chamada isolada.

us5_2.txt

- Script executado pelo EasyAccept através da classe Facade.
- Verifica comportamento de fora para dentro, sem acessar services diretamente.
- Exige que métodos públicos, mensagens e formatos permaneçam compatíveis.
- Ajuda a detectar regressões em regras de negócio e persistência.
- O teste representa uma fatia de uso do sistema, não apenas uma chamada isolada.

us6_1.txt

- Script executado pelo EasyAccept através da classe Facade.
- Verifica comportamento de fora para dentro, sem acessar services diretamente.
- Exige que métodos públicos, mensagens e formatos permaneçam compatíveis.
- Ajuda a detectar regressões em regras de negócio e persistência.
- O teste representa uma fatia de uso do sistema, não apenas uma chamada isolada.

us6_2.txt

- Script executado pelo EasyAccept através da classe Facade.
- Verifica comportamento de fora para dentro, sem acessar services diretamente.
- Exige que métodos públicos, mensagens e formatos permaneçam compatíveis.
- Ajuda a detectar regressões em regras de negócio e persistência.
- O teste representa uma fatia de uso do sistema, não apenas uma chamada isolada.

us7_1.txt

- Script executado pelo EasyAccept através da classe Facade.
- Verifica comportamento de fora para dentro, sem acessar services diretamente.
- Exige que métodos públicos, mensagens e formatos permaneçam compatíveis.
- Ajuda a detectar regressões em regras de negócio e persistência.
- O teste representa uma fatia de uso do sistema, não apenas uma chamada isolada.

us7_2.txt

- Script executado pelo EasyAccept através da classe Facade.
- Verifica comportamento de fora para dentro, sem acessar services diretamente.
- Exige que métodos públicos, mensagens e formatos permaneçam compatíveis.
- Ajuda a detectar regressões em regras de negócio e persistência.
- O teste representa uma fatia de uso do sistema, não apenas uma chamada isolada.

us8_1.txt

- Script executado pelo EasyAccept através da classe Facade.
- Verifica comportamento de fora para dentro, sem acessar services diretamente.
- Exige que métodos públicos, mensagens e formatos permaneçam compatíveis.
- Ajuda a detectar regressões em regras de negócio e persistência.
- O teste representa uma fatia de uso do sistema, não apenas uma chamada isolada.

us8_2.txt

- Script executado pelo EasyAccept através da classe Facade.
- Verifica comportamento de fora para dentro, sem acessar services diretamente.
- Exige que métodos públicos, mensagens e formatos permaneçam compatíveis.
- Ajuda a detectar regressões em regras de negócio e persistência.
- O teste representa uma fatia de uso do sistema, não apenas uma chamada isolada.

Análise de qualidade do design

Coesão

- Cada service possui um assunto principal.
- Cada repository possui uma família de dados principal.
- Os models representam entidades reconhecíveis do domínio.

- A Facade se limita à entrada e à montagem das dependências.

Comentário técnico:

- Esse aspecto melhora a leitura do projeto e reduz o risco de mudanças pequenas causarem efeitos inesperados.
- A qualidade não vem de usar muitas camadas, mas de cada camada ter um motivo claro para existir.

Acoplamento

- A Facade conhece os services, mas os testes não conhecem a estrutura interna.
- Os services conhecem repositories e outros services necessários.
- Repositories não conhecem services.
- Models não dependem de repositories.

Comentário técnico:

- Esse aspecto melhora a leitura do projeto e reduz o risco de mudanças pequenas causarem efeitos inesperados.
- A qualidade não vem de usar muitas camadas, mas de cada camada ter um motivo claro para existir.

Encapsulamento

- Os atributos dos models são privados.
- O acesso ocorre por getters e setters.
- As listas são obtidas por métodos próprios.
- O comportamento específico de subclasses fica nas próprias subclasses.

Comentário técnico:

- Esse aspecto melhora a leitura do projeto e reduz o risco de mudanças pequenas causarem efeitos inesperados.
- A qualidade não vem de usar muitas camadas, mas de cada camada ter um motivo claro para existir.

Evolução

- Nova persistência tende a afetar repositories.
- Nova regra de pedido tende a afetar PedidoService.
- Novo tipo de empresa exige model e criação adequada.
- Novas mensagens de erro devem ser adicionadas como exceções específicas.

Comentário técnico:

- Esse aspecto melhora a leitura do projeto e reduz o risco de mudanças pequenas causarem efeitos inesperados.
- A qualidade não vem de usar muitas camadas, mas de cada camada ter um motivo claro para existir.

Testabilidade

- A Facade oferece uma entrada única para aceitação.
- Services ficam testáveis por comportamento.
- Repositories podem ser avaliados por estado carregado e salvo.
- A separação torna mais fácil localizar a origem de falhas.

Comentário técnico:

- Esse aspecto melhora a leitura do projeto e reduz o risco de mudanças pequenas causarem efeitos inesperados.
- A qualidade não vem de usar muitas camadas, mas de cada camada ter um motivo claro para existir.

Detalhamento complementar por user story

US1 - Criação de contas

A criação de contas é a base do sistema, porque todas as outras operações dependem de algum usuário existente. O fluxo começa na Facade, mas a validação fica concentrada no UsuarioService. Cliente, dono de empresa e entregador compartilham os campos comuns de Usuario. As diferenças aparecem nas subclasses, como CPF no DonoDeEmpresa e veículo/placa no Entregador.

O UsuarioRepository participa gerando ids, conferindo email duplicado, conferindo placa duplicada e guardando a lista em memória. Essa user story também valida o login, que depende da busca por email e senha. O ponto técnico principal é separar criação de objeto, validação de entrada e armazenamento. Se essa separação fosse quebrada, regras de conta ficariam misturadas com XML ou com detalhes do EasyAccept.

US2 - Criação de Restaurantes

A criação de restaurantes introduz a relação entre usuário dono e empresa. EmpresaService precisa consultar UsuarioService para saber se o id informado realmente pertence a um dono de empresa. Restaurante é uma especialização de Empresa e adiciona o campo tipoCozinha. EmpresaRepository gera o id e guarda a empresa criada.

A regra de duplicidade também fica no EmpresaService, porque ela depende de dono, nome e endereço. O retorno de atributos passa pelo model Empresa, e o atributo tipoCozinha é respondido pelo próprio Restaurante. Essa organização reduz dependência de classe concreta no service. O resultado é um fluxo em que a regra de criação fica clara e a entidade mantém seus dados específicos.

US3 - Criação de Produtos

Produtos pertencem a empresas e por isso carregam empresald. ProdutoService valida nome, valor e categoria antes de criar ou editar. ProdutoRepository faz a busca por nome e empresa, que é usada para impedir duplicidade dentro da mesma empresa. O valor do produto é formatado com duas casas quando consultado.

A empresa associada ao produto é consultada via EmpresaService. Essa user story mostra bem a diferença entre dado armazenado e regra de negócio. O repository sabe encontrar produtos, mas não decide se um valor negativo é permitido. Essa decisão fica no service, onde a regra é mais fácil de localizar.

US4 - Criação de Pedidos

PedidoService coordena uma das partes mais importantes do sistema. Ele precisa validar o usuário, validar o pedido aberto duplicado e validar os produtos adicionados. Um dono de empresa não pode abrir pedido, então PedidoService consulta UsuarioService. Produto só pode entrar no pedido se pertencer à mesma empresa do pedido.

PedidoRepository gera o número sequencial e mantém a lista de pedidos. O pedido nasce aberto e muda de estado quando fecharPedido é chamado. O cálculo de valor percorre os produtos do pedido e soma os valores atuais. Essa história conecta usuários, empresas, produtos e pedidos, então é uma das melhores para testar a arquitetura em camadas.

US5 - Criação de Mercados

Mercado é uma especialização de Empresa com horário de funcionamento. EmpresaService valida tipo, nome, endereço, horário de abertura, horário de fechamento e tipoMercado. O formato do horário precisa ser controlado antes de alterar o model. Mercado implementa a alteração de funcionamento porque é a empresa que possui abre e fecha.

Se o id não representa um mercado, a operação falha com exceção específica. Os atributos abre, fecha e tipoMercado são respondidos pelo próprio Mercado. Essa história reforça o uso de polimorfismo no domínio.

A regra de formato continua no service, enquanto o estado final fica no model.

US6 - Criação de Farmacia

Farmacia adiciona os campos aberto24Horas e numeroFuncionarios. EmpresaService valida o tipo farmacia e os dados comuns de criação. A própria Farmacia responde seus atributos específicos. PedidoService usa a informação de que uma empresa é farmácia para priorizar pedidos quando um entregador busca trabalho.

Essa prioridade é uma regra de negócio, não um detalhe de persistência. Por isso ela fica no service e não no repository. A user story mostra que uma subclasse pode ter impacto além da simples consulta de atributo. Mesmo assim, a decisão de prioridade continua isolada no fluxo de pedido.

US7 - Criação de Entregador

Entregador é criado pelo UsuarioService e depois associado a empresas pelo EmpresaService. O cadastro de

entregador em empresa usa ids, então a empresa guarda apenas a referência ao usuário.

UsuarioService

identifica se um usuário é entregador por comportamento do model, e não por instanceof.

EmpresaService

evita cadastrar usuário comum como entregador.

A listagem de entregadores consulta UsuarioService para transformar ids em emails. A listagem de empresas

do entregador percorre as empresas cadastradas e verifica o vínculo. Essa história liga duas áreas do domínio: usuários e empresas. O vínculo fica na Empresa, porque é a empresa que sabe quais entregadores

estão associados a ela.

US8 - Sistema de entregas

EntregaService fecha o ciclo operacional do MyFood. Para criar entrega, o pedido precisa estar pronto. O

entregador precisa ser válido e não pode estar em outra entrega ativa. Quando a entrega é criada, PedidoService muda o estado do pedido para entregando.

Quando entregar é chamado, PedidoService muda o estado para entregue. EntregaRepository mantém o

histórico das entregas criadas. O destino pode vir informado ou pode ser preenchido com o endereço do

cliente. Essa user story depende da integração correta entre pedido, usuário, empresa, produto e entrega.

Por isso ela é a validação mais completa do desenho arquitetural.

Conclusão técnica

O sistema MyFood usa uma arquitetura em camadas simples e adequada ao tamanho do projeto. A Facade

fornece a interface pública exigida pelo EasyAccept. Os Services concentram regras de negócio e validações. Os Repositories isolam armazenamento em memória e persistência XML.

Os Models representam o domínio e carregam comportamento polimórfico quando a resposta depende da especialização da entidade. As Exceptions mantêm o contrato de falhas esperado pelos testes e separam regra de negócio de erro de persistência. Os padrões Facade e Repository foram aplicados por necessidade real do projeto, não apenas por formalidade. Facade resolveu a necessidade de uma entrada única sem concentrar regra.

Repository resolveu a necessidade de persistir dados sem misturar XML com validação de domínio. A organização atual permite entender o fluxo de cada user story a partir dos componentes envolvidos. Também permite trocar partes internas com impacto limitado, desde que o contrato externo da Facade seja preservado. Com isso, o projeto fica coerente com os objetivos da disciplina e com a execução dos testes de aceitação.